Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа № 2

Основы объектно-ориентированного программирования

по дисциплине «Программирование»

Вариант № 753300

Выполнил:

Фещенко А.Э.

Группа P3117

Преподаватель:
Кулинич Я.В.

Санкт-Петербург

2026

Содержание

1. Текст задания.....3
2. Диаграмма классов реализованной объектной модели.....5
3. Исходный код программы.....6
4. Результат работы программы.....13
5. Выводы.....15

1. Текст задания

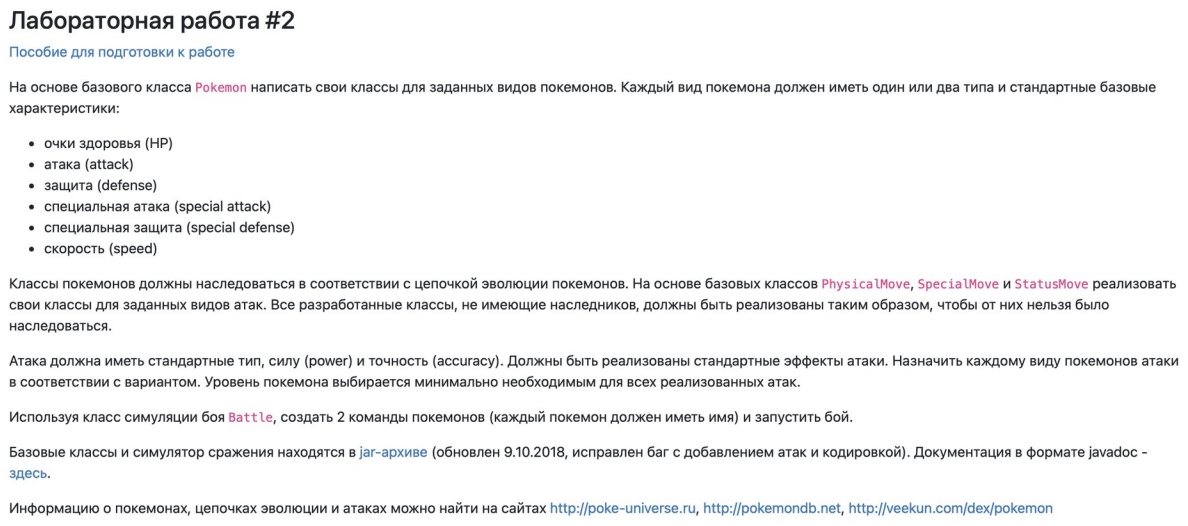
На основе базового класса Pokemon написать свои классы для заданных видов покемонов. Каждый вид покемона должен иметь один или два типа и стандартные базовые характеристики: очки здоровья (HP), атака (attack), защита (defense), специальная атака (special attack), специальная защита (special defense), скорость (speed).

Классы покемонов должны наследоваться в соответствии с цепочкой эволюции покемонов. На основе базовых классов PhysicalMove, SpecialMove и StatusMove реализовать свои классы для заданных видов атак. Все разработанные классы, не имеющие наследников, должны быть реализованы таким образом, чтобы от них нельзя было наследоваться.

Атака должна иметь стандартные тип, силу (power) и точность (accuracy). Должны быть реализованы стандартные эффекты атаки. Назначить каждому виду покемонов атаки в соответствии с вариантом. Уровень покемона выбирается минимально необходимым для всех реализованных атак.

Используя класс симуляции боя Battle, создать 2 команды покемонов (каждый покемон должен иметь имя) и запустить бой.

Базовые классы и симулятор сражения находятся в jar-архиве Pokemon.jar; распаковывать его не нужно, его необходимо подключать к программе как внешнюю библиотеку при компиляции и запуске.



Лабораторная работа #2

Пособие для подготовки к работе

На основе базового класса `Pokemon` написать свои классы для заданных видов покемонов. Каждый вид покемона должен иметь один или два типа и стандартные базовые характеристики:

- очки здоровья (HP)
- атака (attack)
- защита (defense)
- специальная атака (special attack)
- специальная защита (special defense)
- скорость (speed)

Классы покемонов должны наследоваться в соответствии с цепочкой эволюции покемонов. На основе базовых классов `PhysicalMove`, `SpecialMove` и `StatusMove` реализовать свои классы для заданных видов атак. Все разработанные классы, не имеющие наследников, должны быть реализованы таким образом, чтобы от них нельзя было наследоваться.

Атака должна иметь стандартные тип, силу (power) и точность (accuracy). Должны быть реализованы стандартные эффекты атаки. Назначить каждому виду покемонов атаки в соответствии с вариантом. Уровень покемона выбирается минимально необходимым для всех реализованных атак.

Используя класс симуляции боя `Battle`, создать 2 команды покемонов (каждый покемон должен иметь имя) и запустить бой.

Базовые классы и симулятор сражения находятся в jar-архиве (обновлен 9.10.2018, исправлен баг с добавлением атак и кодировкой). Документация в формате javadoc - здесь.

Информацию о покемонах, цепочках эволюции и атаках можно найти на сайтах http://poke-universe.ru, http://pokemondb.net, http://veekun.com/dex/pokemon

Рисунок 1 — Текст задания

Вариант 753300:

(Программирование) Лабораторная работа №2

Введите вариант:

753300

Ваши покемоны:

Thundurus



Атаки:
Focus Blast
Charge Beam
Thunder
Confide

Golett



Атаки:
Shadow Ball
Low Sweep
Rock Polish

Golurk



Атаки:
Shadow Ball
Low Sweep
Rock Polish
Flash Cannon

Flabebe



Атаки:
Dazzling Gleam
Facade

Floette



Атаки:
Dazzling Gleam
Facade
Vine Whip

Florges



Атаки:
Dazzling Gleam
Facade
Vine Whip
Disarming Voice

Рисунок 2 — Покемоны и атаки варианта 753300

2. Диаграмма классов реализованной объектной модели

Диаграмма построена в draw.io. Серым цветом показаны классы библиотеки Pokemon.jar, от которых наследуются классы программы; белым — классы, написанные в работе (пакеты pokemons и moves). Стрелки с незакрашенным треугольником обозначают наследование (extends), стереотип «final» — класс, от которого нельзя наследоваться, «abstract» — абстрактный класс.

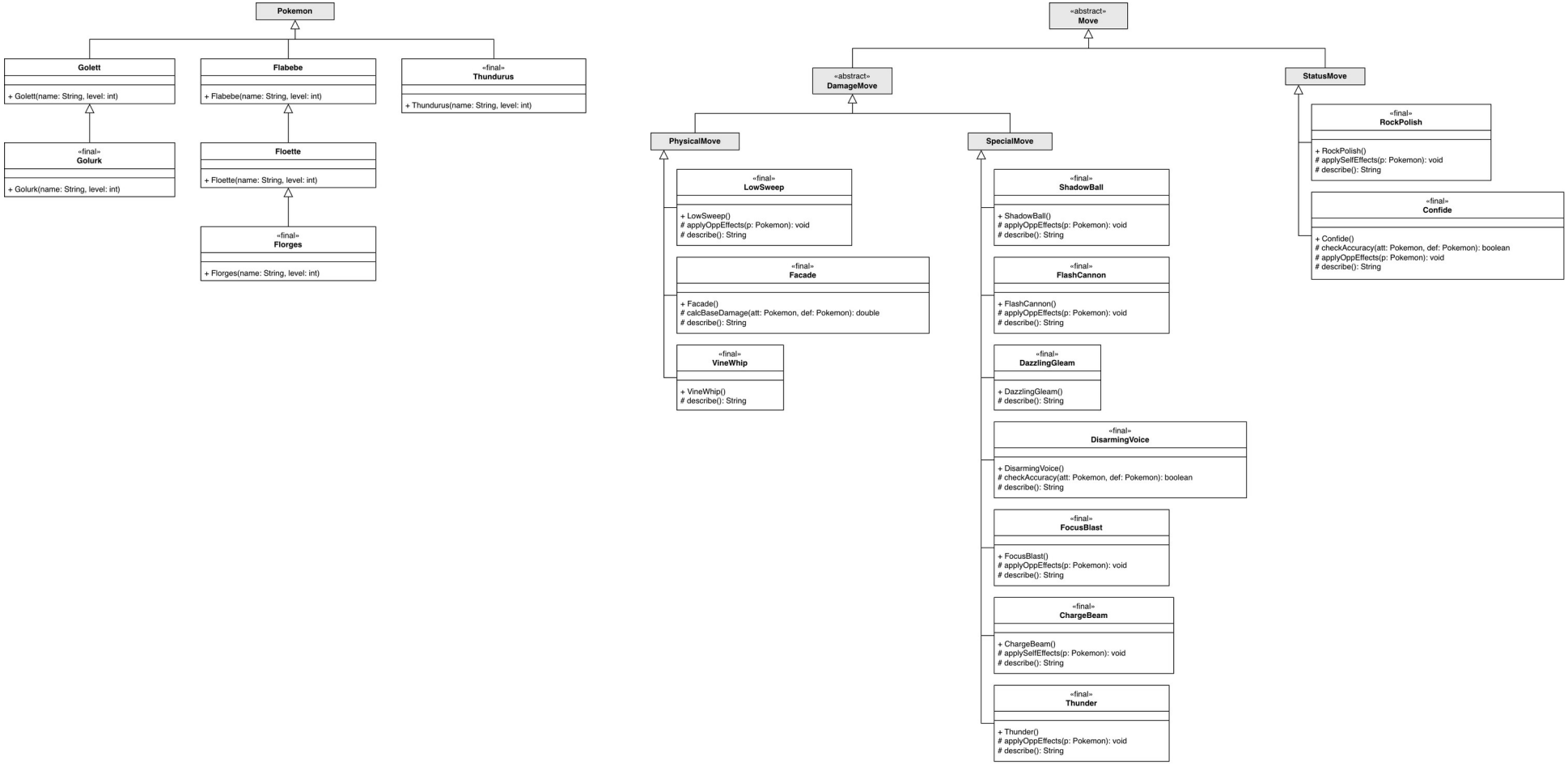


Рисунок 3 — Диаграмма классов

3. Исходный код программы

Программа состоит из трёх пакетов: lab2 (класс Main с созданием команд и запуском боя), pokemons (классы покемонов) и moves (классы атак).

Файл src/lab2/Main.java:

```
package lab2;

import pokemons.*;
import ru.ifmo.se.pokemon.*;

public class Main {
    public static void main(String[] args) {
        Battle b = new Battle();
        Pokemon p1 = new Golett("Mini Rock", 1);
        Pokemon p2 = new Golurk("Big Boss", 43);
        Pokemon p3 = new Floette("Flower Knight", 19);
        Pokemon p4 = new Flabebe("Flower", 1);
        Pokemon p5 = new Florges("Flower Queen", 19);
        Pokemon p6 = new Thundurus("Pokemon God", 1);
        b.addAlly(p1);
        b.addAlly(p2);
        b.addFoe(p3);
        b.addFoe(p4);
        b.addFoe(p5);
        b.addFoe(p6);
        b.go();
    }
}
```

Файл src/pokemons/Golett.java:

```
package pokemons;

import moves.*;
import ru.ifmo.se.pokemon.*;

public class Golett extends Pokemon {
    public Golett(String name, int level) {
        super(name,level);
        setType(Type.GROUND, Type.GHOST);
        setStats(59, 74, 50, 35, 50, 35);
        setMove(new ShadowBall(), new LowSweep(), new RockPolish());
    }
}
```

Файл src/pokemons/Golurk.java:

```
package pokemons;

import moves.*;
import ru.ifmo.se.pokemon.*;

public final class Golurk extends Golett {
    public Golurk(String name, int level) {
        super(name, level);
        setStats(89, 124, 80, 55, 80, 55);
        addMove(new FlashCannon());
    }
}
```

Файл src/pokemons/Flabebe.java:

```
package pokemons;
```

```
import moves.*;
import ru.ifmo.se.pokemon.*;

public class Flabebe extends Pokemon {
    public Flabebe(String name, int level) {
            super(name, level);
            setType(Type.FAIRY);
            setStats(44, 38, 39, 61, 79, 42);
            setMove(new DazzlingGleam(), new Facade());
    }
}
```

Файл src/pokemons/Floette.java:

```
package pokemons;

import moves.*;
import ru.ifmo.se.pokemon.*;

public class Floette extends Flabebe {
    public Floette(String name, int level) {
        super(name, level);
        setStats(54, 45, 47, 75, 98, 52);
        addMove(new VineWhip());
    }
}
```

Файл src/pokemons/Florges.java:

```
package pokemons;

import moves.*;
import ru.ifmo.se.pokemon.*;

public final class Florges extends Floette {
    public Florges(String name, int level) {
        super(name, level);
        setStats(78, 65, 68, 112, 154, 75);
        addMove(new DisarmingVoice());
    }
}
```

Файл src/pokemons/Thundurus.java:

```
package pokemons;

import moves.*;
import ru.ifmo.se.pokemon.*;

public final class Thundurus extends Pokemon{
    public Thundurus(String name, int level) {
        super(name, level);
        setType(Type.ELECTRIC, Type.FLYING);
        setStats(79, 115, 70, 125, 80, 111);
        setMove(new FocusBlast(), new ChargeBeam(), new Thunder(), new Confide());
    }
}
```

Файл src/moves/LowSweep.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class LowSweep extends PhysicalMove {
    public LowSweep() {
```

```
        super(Type.FIGHTING, 65, 1);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        p.setMod(Stat.SPEED, -1);
    }

    @Override
    protected String describe() {
        return "делает подсечку Low Sweep";
    }
}
```

Файл src/moves/Facade.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class Facade extends PhysicalMove {
    public Facade() {
        super(Type.NORMAL, 70, 1);
    }

    @Override
    protected double calcBaseDamage(Pokemon att, Pokemon def) {
        double damage = super.calcBaseDamage(att, def);
        Status s = att.getCondition();
        if (s == Status.BURN || s == Status.POISON || s == Status.PARALYZE) {
            damage = damage * 2;
        }
        return damage;
    }

    @Override
    protected String describe() {
        return "бьёт Facade";
    }
}
```

Файл src/moves/VineWhip.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class VineWhip extends PhysicalMove {
    public VineWhip() {
        super(Type.GRASS, 45, 1);
    }

    @Override
    protected String describe() {
        return "хлещет лозой Vine Whip";
    }
}
```

Файл src/moves/ShadowBall.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class ShadowBall extends SpecialMove{
public ShadowBall() {
        super(Type.GHOST, 80, 1);
```

```
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (Math.random()<0.2) {
            p.setMod(Stat.SPECIAL_DEFENSE, -1);
        }
    }

    @Override
    protected String describe() {
        return "бросает Shadow Ball";
    }
}
```

Файл src/moves/FlashCannon.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class FlashCannon extends SpecialMove {
    public FlashCannon() {
        super(Type.STEEL, 80, 1);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (Math.random() < 0.1) {
            p.setMod(Stat.SPECIAL_DEFENSE, -1);
        }
    }

    @Override
    protected String describe() {
        return "стреляет Flash Cannon";
    }
}
```

Файл src/moves/DazzlingGleam.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class DazzlingGleam extends SpecialMove {
    public DazzlingGleam() {
        super(Type.FAIRY, 80, 1);
    }

    @Override
    protected String describe() {
        return "ослепляет Dazzling Gleam";
    }
}
```

Файл src/moves/DisarmingVoice.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class DisarmingVoice extends SpecialMove {
    public DisarmingVoice() {
        super(Type.FAIRY, 40, 1);
    }
```

```
    @Override
    protected boolean checkAccuracy(Pokemon att, Pokemon def) {
        return true;
    }

    @Override
    protected String describe() {
        return "кричит Disarming Voice";
    }
}
```

Файл src/moves/FocusBlast.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class FocusBlast extends SpecialMove {
    public FocusBlast() {
        super(Type.FIGHTING, 120, 0.7);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (Math.random() < 0.1) {
            p.setMod(Stat.SPECIAL_DEFENSE, -1);
        }
    }

    @Override
    protected String describe() {
        return "запускает Focus Blast";
    }
}
```

Файл src/moves/ChargeBeam.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class ChargeBeam extends SpecialMove {
    public ChargeBeam() {
        super(Type.ELECTRIC, 50, 0.9);
    }

    @Override
    protected void applySelfEffects(Pokemon p) {
        if (Math.random() < 0.7) {
            p.setMod(Stat.SPECIAL_ATTACK, 1);
        }
    }

    @Override
    protected String describe() {
        return "стреляет лучом Charge Beam";
    }
}
```

Файл src/moves/Thunder.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class Thunder extends SpecialMove {
    public Thunder() {
```

```
        super(Type.ELECTRIC, 110, 0.7);
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        if (Math.random() < 0.3) {
            Effect.paralyze(p);
        }
    }

    @Override
    protected String describe() {
        return "вызывает гром Thunder";
    }
}
```

Файл src/moves/RockPolish.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class RockPolish extends StatusMove {
    public RockPolish() {
        super(Type.ROCK, 0, 1);
    }

    @Override
    protected void applySelfEffects(Pokemon p) {
        p.setMod(Stat.SPEED, 2);
    }

    @Override
    protected String describe() {
        return "полирует себя Rock Polish";
    }
}
```

Файл src/moves/Confide.java:

```
package moves;

import ru.ifmo.se.pokemon.*;

public final class Confide extends StatusMove {
    public Confide() {
        super(Type.NORMAL, 0, 1);
    }

    @Override
    protected boolean checkAccuracy(Pokemon att, Pokemon def) {
        return true;
    }

    @Override
    protected void applyOppEffects(Pokemon p) {
        p.setMod(Stat.SPECIAL_ATTACK, -1);
    }

    @Override
    protected String describe() {
        return "шепчет секрет Confide";
    }
}
```

Компиляция, сборка исполняемого jar-архива и запуск на сервере helios (архив Pokemon.jar подключается через атрибут Class-Path манифеста, без распаковки):

```
scp -P 2222 -r src s561306@helios.cs.ifmo.ru:~/proglab2/
javac -cp Pokemon.jar -sourcepath src -d out src/lab2/Main.java
printf 'Main-Class: lab2.Main\nClass-Path: Pokemon.jar\n' > MANIFEST.MF
jar cfm lab2.jar MANIFEST.MF -C out .
java -jar lab2.jar
```

4. Результат работы программы

Сборка и запуск программы на сервере helios:

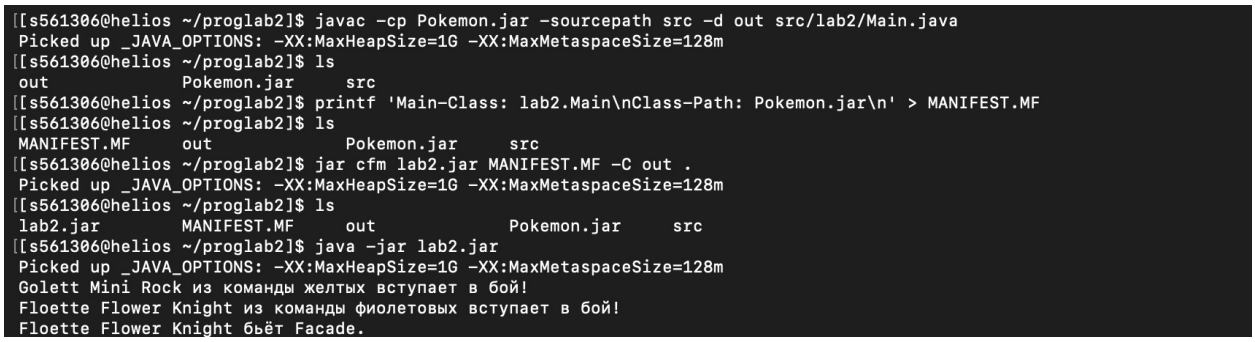


Рисунок 4 — Компиляция, сборка lab2.jar и запуск на сервере helios

Полный вывод программы (команда желтых: Golett «Mini Rock», Golurk «Big Boss»; команда фиолетовых: Floette «Flower Knight», Flabebe «Flower», Florges «Flower Queen», Thundurus «Pokemon God»). Ход боя зависит от случайных чисел, поэтому при каждом запуске вывод отличается:

```
Golett Mini Rock из команды желтых вступает в бой!
Floette Flower Knight из команды фиолетовых вступает в бой!
Floette Flower Knight бьёт Facade.
Golett Mini Rock теряет 1 здоровья.
Golett Mini Rock не замечает воздействие типа NORMAL

Golett Mini Rock полирует себя Rock Polish.
Golett Mini Rock увеличивает скорость.

Floette Flower Knight бьёт Facade.
Golett Mini Rock теряет 1 здоровья.
Golett Mini Rock не замечает воздействие типа NORMAL

Golett Mini Rock делает подсечку Low Sweep.
Floette Flower Knight теряет 2 здоровья.
Floette Flower Knight уменьшает скорость.

Floette Flower Knight бьёт Facade.
Golett Mini Rock теряет 1 здоровья.
Golett Mini Rock не замечает воздействие типа NORMAL

Golett Mini Rock полирует себя Rock Polish.
Golett Mini Rock увеличивает скорость.

Floette Flower Knight бьёт Facade.
Golett Mini Rock теряет 1 здоровья.
Golett Mini Rock не замечает воздействие типа NORMAL

Golett Mini Rock полирует себя Rock Polish.
Golett Mini Rock увеличивает скорость.

Floette Flower Knight ослепляет Dazzling Gleam.
Golett Mini Rock теряет 86 здоровья.
Golett Mini Rock теряет сознание.
Golurk Big Boss из команды желтых вступает в бой!
Golurk Big Boss бросает Shadow Ball.
Floette Flower Knight теряет 23 здоровья.

Floette Flower Knight бьёт Facade.
Golurk Big Boss теряет 1 здоровья.
Golurk Big Boss не замечает воздействие типа NORMAL

Golurk Big Boss делает подсечку Low Sweep.
```

```
Floette Flower Knight теряет 25 здоровья.
Floette Flower Knight уменьшает скорость.

Floette Flower Knight бьёт Facade.
Критический удар!
Golurk Big Boss теряет 1 здоровья.
Golurk Big Boss не замечает воздействие типа NORMAL

Floette Flower Knight хлещет лозой Vine Whip.
Критический удар!
Golurk Big Boss теряет 17 здоровья.

Golurk Big Boss стреляет Flash Cannon.
Floette Flower Knight теряет 9 здоровья.
Floette Flower Knight теряет сознание.
Flabebe Flower из команды фиолетовых вступает в бой!
Golurk Big Boss полирует себя Rock Polish.
Golurk Big Boss увеличивает скорость.

Flabebe Flower ослепляет Dazzling Gleam.
Golurk Big Boss теряет 5 здоровья.

Golurk Big Boss бросает Shadow Ball.
Flabebe Flower теряет 143 здоровья.
Flabebe Flower теряет сознание.
Florges Flower Queen из команды фиолетовых вступает в бой!
Golurk Big Boss делает подсечку Low Sweep.
Florges Flower Queen теряет 17 здоровья.
Florges Flower Queen уменьшает скорость.

Florges Flower Queen кричит Disarming Voice.
Golurk Big Boss теряет 6 здоровья.

Golurk Big Boss стреляет Flash Cannon.
Florges Flower Queen теряет 11 здоровья.

Florges Flower Queen кричит Disarming Voice.
Golurk Big Boss теряет 9 здоровья.

Golurk Big Boss бросает Shadow Ball.
Критический удар!
Florges Flower Queen теряет 34 здоровья.
Florges Flower Queen уменьшает специальную защиту.
Florges Flower Queen теряет сознание.
Thundurus Pokemon God из команды фиолетовых вступает в бой!
Golurk Big Boss делает подсечку Low Sweep.
Thundurus Pokemon God теряет 117 здоровья.
Thundurus Pokemon God уменьшает скорость.
Thundurus Pokemon God теряет сознание.
В команде фиолетовых не осталось покемонов.
Команда желтых побеждает в этом бою!
```

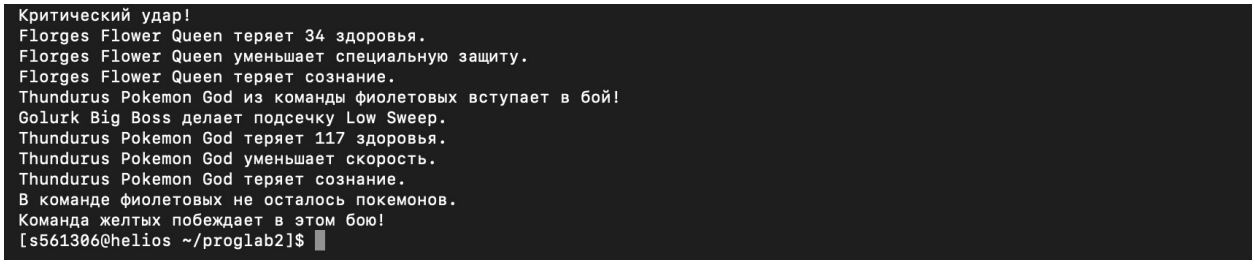


Рисунок 5 — Окончание боя на сервере helios

5. Выводы

В ходе выполнения лабораторной работы я познакомился с основами объектно-ориентированного программирования на Java на примере готовой библиотеки Pokemon.jar: научился создавать собственные классы на основе чужих базовых классов и подключать внешний jar-архив к программе через classpath при компиляции и через атрибут Class-Path манифеста при запуске, не распаковывая его.

Классы покемонов построены по цепочкам эволюции с помощью наследования: Golurk расширяет Golett, Floette — Flabebe, Florges — Floette. Наследник вызывает конструктор родителя через super(name, level), получает от него тип и атаки и только добавляет новую атаку через addMove и меняет базовые характеристики. Классы, у которых нет наследников (Golurk, Florges, Thundurus и все атаки), объявлены final.

Атаки реализованы как наследники PhysicalMove, SpecialMove и StatusMove. Их поведение изменяется переопределением protected-методов базового класса: describe задаёт сообщение, applyOppEffects и applySelfEffects — эффекты на противника и на себя, checkAccuracy позволяет атакам Confide и Disarming Voice никогда не промахиваться, а calcBaseDamage удваивает урон Facade, если атакующий отравлен, обожжён или парализован. Это пример полиморфизма: Battle вызывает одни и те же методы, а выполняется код конкретного класса атаки.

Также я разобрался, зачем библиотека делает многие методы final и protected: это инкапсуляция — механику боя нельзя сломать, а наследнику доступно ровно то, что нужно для настройки. Программа собрана в исполняемый jar-архив и запущена на сервере helios; команда желтых победила, но, судя по 117 урона по Thundurus за один Low Sweep, богам покемонов тоже стоит качать уровень.